



Phân tích và đánh giá dữ liệu kỹ thuật bóng đá Từ mùa giải 2023/24 và 2024/25

Group 7

- Nguyễn Tiến Long – SE200961
- Võ Minh Nhật Quang – SE202066
- Nguyễn Phan Nhật Đăng – SE201827
- Huỳnh Minh Hiệp – SE201835
- Lecture: Đoàn Nguyễn Thành Hoà

-MỤC LỤC:

- **CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN ·**
- **CHƯƠNG 2: PHÂN TÍCH DỮ LIỆU ·**
- **CHƯƠNG 3: THU THẬP VÀ XỬ LÝ DỮ LIỆU ·**
- **CHƯƠNG 4: ANALYSIS WITH SQL (PHÂN TÍCH BẰNG SQL) ·**
- **CHƯƠNG 5: ANALYSIS WITH PYTHON (PHÂN TÍCH BẰNG PYTHON)·**
- **CHƯƠNG 6: VISUALIZATION (TRỰC QUAN HÓA) ·**
- **CHƯƠNG 7: REGRESSION ANALYSIS (PHÂN TÍCH HỒI QUY) ·**
- **CHƯƠNG 8: ANALYSIS TOOL (CÔNG CỤ PHÂN TÍCH) ·**

CHƯƠNG 1: GIỚI THIỆU TỔNG QUAN

Báo cáo này trình bày kết quả phân tích chuyên sâu về dữ liệu thống kê của các trận đấu bóng đá trong các mùa giải gần đây. Mục tiêu chính là chuyển đổi dữ liệu thô từ cấp độ trận đấu sang các **chỉ số hiệu suất then chốt (KPIs)** có ý nghĩa ở cấp độ đội, sau đó sử dụng các công cụ phân tích tiên tiến để định lượng các yếu tố dẫn đến chiến thắng.

Mục tiêu Cốt lõi: Định lượng thành công

Chúng tôi tìm cách trả lời câu hỏi cốt lõi: **Yếu tố nào (Hiệu suất Sút, Số thẻ phạt, Hiệu số Bàn thắng) có ảnh hưởng lớn nhất đến Tỷ lệ Thắng (Win Rate) của một đội bóng?**

Quá trình phân tích sẽ tuân theo các giai đoạn sau:

1. **Data Preprocessing (Tiền xử lý):** Sử dụng các kỹ thuật Python/Pandas để làm sạch dữ liệu và sử dụng **SQL UNION ALL** để chuẩn hóa dữ liệu Sân nhà/Sân khách thành một định dạng duy nhất.
2. **Exploratory Data Analysis (EDA):** Tính toán các chỉ số thống kê mô tả (trung bình, tổng, hiệu suất) và trực quan hóa các mối quan hệ (ví dụ: Biểu đồ Phân tán giữa Sút trúng đích và Tỷ lệ thắng).
3. **Regression Analysis (Phân tích Hồi quy):** Xây dựng mô hình **Hồi quy Tuyến tính Đa biến** để xác định mức độ ảnh hưởng của từng KPI đến Tỷ lệ Thắng.

Kết quả của báo cáo này sẽ cung cấp bằng chứng dựa trên dữ liệu để đưa ra các đề xuất chiến thuật và đánh giá đội bóng hiệu quả.

CHƯƠNG 2: PHÂN TÍCH DỮ LIỆU

2.1. Mục tiêu nghiên cứu

- + Xác định Đội mạnh nhất trong tập dữ liệu dựa trên tổng số điểm và chuỗi phong độ.
- + Đánh giá Hiệu quả dứt điểm thông qua tỷ lệ sút trúng đích của từng đội.
- + Phân tích Tính kỷ luật của các đội qua số lượng thẻ phạt trung bình.
- + So sánh Lợi thế sân nhà/sân khách bằng cách phân tích tỷ lệ thắng.
- + Xác định đội có Tính ổn định (an toàn) cao nhất thông qua số trận hòa.

2.2. Phạm vi

Báo cáo tập trung vào dữ liệu thống kê trận đấu của các đội bóng trong [tên giải đấu/các giải đấu] từ mùa giải 2023/24 đến 2024/25.

2.3. Cách tiếp cận

Sử dụng các công cụ SQL (để tổng hợp) và Python (để phân tích chuyên sâu và hồi quy) để chuyển đổi dữ liệu thô thành các chỉ số hiệu suất có ý nghĩa.

CHƯƠNG 3: THU THẬP VÀ PHÂN TÍCH DỮ LIỆU

3.1. Thu thập và làm sạch dữ liệu

- Nguồn dữ liệu: dữ liệu được thu thập dựa trên thống kê trận đấu của các đội từ mùa giải 2023/24 đến 2024/25 trên Kaggle và được lưu lại dưới dạng file CSV và lưu trữ trong thư mục dự án để xử lý sơ bộ

3.2 Quy trình làm sạch

3.2.1: Chuẩn hoá và đổi tên cột

- Để tạo điều kiện thuận lợi cho việc xử lý dữ liệu hiệu quả, tất cả tên cột đã được chuẩn hóa.
- Chúng tôi đã chuyển đổi tên cột thành chữ thường, thay thế khoảng trắng bằng dấu gạch dưới (underscore), và rút gọn các tên biến dài để tiện sử dụng.

Ví dụ: cột FullTimeHomeGoals đã được đổi tên thành home_goals, và HomeShotsOnTarget được rút gọn thành home_sot.

Đoạn mã Python dưới đây minh họa quá trình chuẩn hóa và đổi tên này.

```

1 import pandas as pd
2
3 # --- 1. Đọc dữ liệu gốc ---
4 df = pd.read_csv("team_stats.csv")
5
6 # --- 2. Chuẩn hóa tên cột (normalize column names) ---
7 df.columns = (
8     df.columns
9     .str.strip()          # bỏ khoảng trắng 2 đầu
10    .str.lower()          # viết thường hết
11    .str.replace(' ', '_') # thay space = _
12    .str.replace('-', '_') # thay dấu - = _
13    .str.replace(r'[^\w-z0-9_]', '', regex=True) # bỏ ký tự đặc biệt
14 )
15
16 # --- 3. Đổi tên cột dài, trùng, hoặc khó đọc sang dạng ngắn gọn ---
17 rename_dict = {
18     'season': 'season',
19     'matchdate': 'match_date',
20     'hometeam': 'home_team',
21     'awayteam': 'away_team',
22     'fulltimehomegoals': 'home_goals',
23     'fulltimeawaygoals': 'away_goals',
24     'fulltimeresult': 'ft_result',
25     'halftimehomegoals': 'ht_home_goals',
26     'halftimeawaygoals': 'ht_away_goals',
27     'halftimeresult': 'ht_result',
28     'homeshots': 'home_shots',
29     'awayshots': 'away_shots',
30     'homeshotsontarget': 'home_sot',
31     'awayshotsontarget': 'away_sot',
32     'homecorners': 'home_corners',
33     'awaycorners': 'away_corners',
34     'homefouls': 'home_fouls',
35     'awayfouls': 'away_fouls',
36     'homeyellowcards': 'home_yellow',
37     'awayyellowcards': 'away_yellow',
38     'homeredcards': 'home_red',
39     'awayredcards': 'away_red'
40 }
41
42 df = df.rename(columns=rename_dict)
43
44 # --- | Xử lý kiểu dữ liệu (đặc biệt là ngày tháng) ---
45 df["match_date"] = pd.to_datetime(df["match_date"], errors='coerce')

```

Figure 3.2: Python code for column normalization and renaming.

Sau khi chạy xong code, Tên các cột đã được chuẩn hóa (sau khi xử lý)

Original Name	Renamed Column
---------------	----------------

FullTimeHomeGoals	home_goals
FullTimeAwayGoals	away_goals
HomeShotsOnTarget	home_sot
AwayShotsOnTarget	away_sot
HomeYellowCards	home_yellow
AwayYellowCards	away_yellow
HomeRedCards	home_red
AwayRedCards	away_red

Table 3.2: Sample of standardized column names

Với tất cả các tên cột đã được chuẩn hóa, tập dữ liệu đã sẵn sàng cho kỹ thuật đặc trưng chuyên sâu hơn

3.2.2: Chuyển matchdate từ dạng STR sang DATETIME

```
# 2. Xử lý cột Ngày tháng
df["MatchDate"] = pd.to_datetime(df["MatchDate"], errors='coerce')
```

CHƯƠNG 4: PHÂN TÍCH BẰNG SQL

4.1: Import file CSV lên SQL

4.1.1 Giới thiệu Module: Tích hợp Hệ thống (ETL)

Phần mã này thực hiện quá trình **Trích xuất, Biến đổi, và Tải (ETL - Extract, Transform, Load)** dữ liệu thống kê trận đấu từ tệp CSV vào hệ thống cơ sở dữ liệu quan hệ (SQL Server).

Mục tiêu chính là xây dựng cầu nối ổn định và tự động hóa việc đồng bộ hóa dữ liệu phân tích giữa môi trường lập trình Python (Pandas) và cơ sở dữ liệu. Điều này rất quan trọng để đảm bảo dữ liệu đầu vào cho các bước Phân tích SQL và Hồi quy sau này là chính xác và có cấu trúc.

Dưới đây là đoạn code dùng để thực hiện quy trình trên:

```
import pandas as pd
import numpy as np
from sqlalchemy import create_engine
import pyodbc

#====CONFIG=====
server = "RONALDO\\SQLEXPRESS"
database = "FootballDB"
csv_path = "team_stats.csv"
#=====
conn = pyodbc.connect( r"Driver={ODBC Driver 11 for SQL Server};"
    r"Server=RONALDO\\SQLEXPRESS;"
    r"Database=FootballDB;"
    r"Trusted_Connection=yes;")
cursor = conn.cursor()
cursor.execute("SELECT DB_NAME()")
row = cursor.fetchone()
print(" Đang kết nối vào database:", row[0])
# Kết nối SQL
engine = create_engine(
    r"mssql+pyodbc://RONALDO\\SQLEXPRESS/FootballDB?driver=ODBC+Driver+11+for+SQL+Server&trusted_connection=yes"
)
# Đọc CSV
df = pd.read_csv("team_stats.csv")

# Tự tạo bảng & import
df.to_sql("EPL_Matches", engine, if_exists="replace", index=False)
```

4.1.2 Giải thích Chi tiết Quy trình Mã

Phần code này sử dụng thư viện **SQLAlchemy** để tạo một **Engine kết nối** ổn định, loại bỏ các vấn đề không tương thích giữa Pandas và SQL Server.

A. Thiết lập Kết nối (Connection Setup)

Dòng Code	Tác dụng trong hệ thống
<pre>from sqlalchemy import create_engine</pre>	Nhập thư viện để tạo đối tượng Engine. Engine này quản lý chi tiết kết nối cho Pandas, giúp giải quyết lỗi <code>sqlite_master</code> thường gặp.
<pre>connection_uri = f'mssql+pyodbc:///?odbc_connect =...'</pre>	Xây dựng URI Kết nối: Xác định phương thức kết nối (<code>mssql+pyodbc</code>) và truyền chuỗi xác thực ODBC (<code>DRIVER</code> , <code>SERVER</code> , <code>DATABASE</code> , <code>Trusted Connection</code>). Phương pháp này là tiêu chuẩn để đảm bảo tính ổn định trong môi trường Python.
<pre>engine = create_engine(connection_uri)</pre>	Khởi tạo: Tạo Engine kết nối chính thức, sẵn sàng cho việc truyền dữ liệu.

Hàm Tiền xử lý Dữ liệu (*clean_and_prepare_data*)

Phần này chuyển đổi dữ liệu thô từ CSV thành định dạng sẵn sàng cho SQL Server.

Dòng Code	Tác dụng trong hệ thống
-----------	-------------------------

<code>df.columns = df.columns.str.lower(...)</code>	Chuẩn hóa Tên cột: Đảm bảo tất cả tên cột đều tuân thủ quy tắc snake_case và loại bỏ các ký tự đặc biệt, ngăn ngừa lỗi cú pháp SQL và nhất quán hóa dữ liệu.
<code>df["matchdate"] = pd.to_datetime(...)</code>	Chuyển đổi Kiểu dữ liệu: Đảm bảo cột ngày tháng được nhận dạng là datetime, đây là yêu cầu cốt lõi của Database.
<code>df[col] = df[col].fillna(0).astype(int, errors='ignore')</code>	Đảm bảo Tính toàn vẹn số học: Thay thế các giá trị thiếu (NaN) bằng 0 và ép kiểu các cột số (Goals, Cards) thành số nguyên (INT). Thao tác này là bắt buộc để ngăn chặn lỗi "Data Type Conversion Error" khi dữ liệu được đẩy vào SQL Server.

Thực thi và Tải dữ liệu (Execution and Load)

Phần này quản lý việc xóa bảng cũ và tải dữ liệu mới.

Dòng Code	Tác dụng trong hệ thống
<code>connection.execute(text(f"DROP TABLE IF EXISTS {TABLE_NAME}"))</code>	Quản trị Database (DDL): Thực thi lệnh SQL để xóa bảng đích (nếu tồn tại). Điều này đảm bảo mỗi lần chạy là một lần làm sạch, tránh lỗi xung đột cấu trúc hoặc trùng lặp dữ liệu.

<code>connection.commit()</code>	Cam kết Thay đổi: Xác nhận và lưu thay đổi DROP TABLE vĩnh viễn vào Database.
<code>df_clean.to_sql(name=TABLE_NAME, con=engine, ...)</code>	Tải Dữ liệu (Load): Lệnh DML/DDDL chính. Pandas tự động tạo lệnh CREATE TABLE (dựa trên cấu trúc DataFrame đã làm sạch) và thực hiện chèn hàng loạt (Bulk Insert) dữ liệu vào SQL Server.

4.2: PHÂN TÍCH BẢNG SQL

4.2.1 Mục đích:

Mục đích của bước này là sử dụng ngôn ngữ truy vấn có cấu trúc **SQL** để tương tác với cơ sở dữ liệu trận đấu (`team_stats.csv`) nhằm tóm tắt dữ liệu, tìm ra các chỉ số hiệu suất quan trọng, và xác định đội mạnh nhất trong mỗi mùa giải dựa trên hệ thống tính điểm chuẩn.

Theo tiêu chí đánh giá, sinh viên cần trình bày **các câu truy vấn bằng SQL** để giải quyết **ít nhất 5 câu hỏi** về dữ liệu trận đấu.

4.2.2 Các câu truy vấn SQL :

1) Tổng số điểm

Mục đích: Xác định đội mạnh nhất (thay vì chỉ nhìn bàn thắng) bằng cách áp dụng quy tắc tính điểm bóng đá (Thắng=3, Hòa=1, Thua=0)

Dưới đây là câu truy vấn sql dành cho mục đích trên:

```
--3. Tổng số điểm của mỗi đội trong mùa giải
--Mục đích: Xác định đội mạnh nhất (thay vì chỉ nhìn bàn thắng)
select a.Season, a.Team, sum(a.point) as point
from (
    select a.Season, a.HomeTeam Team,
    (COUNT(CASE WHEN a.FullTimeResult = 'H' THEN 1 END)*3 +COUNT(CASE WHEN a.FullTimeResult = 'D' THEN 1 END)) as point
    from epl_match_24 a
    group by a.Season, a.HomeTeam
    union all
    select a.Season, a.AwayTeam Team,
    (COUNT(CASE WHEN a.FullTimeResult = 'A' THEN 1 END)*3 +COUNT(CASE WHEN a.FullTimeResult = 'D' THEN 1 END)) as point
    from epl_match_24 a
    group by a.Season, a.AwayTeam) a
group by a.Season, a.Team
order by sum(a.point) desc
```

2) Hiệu suất sút trúng đích của đội chủ nhà và đội khách

Mục đích: Đánh giá Hiệu quả dứt điểm thông qua tỷ lệ sút trúng đích của từng đội.

```
--5. Hiệu suất sút trúng đích của đội chủ nhà và đội khách
--Mục đích: Đánh giá hiệu quả dứt điểm.
select a.Season, Format(sum(a.HomeShotsOnTarget)*1.0/sum(a.HomeShots), 'P2') as Performance,
    (sum(a.AwayShotsOnTarget)*1.0/sum(a.AwayShots)) as Away_Performance
from epl_match_24 a
group by a.Season
```

3) Tỷ lệ thẻ vàng và thẻ đỏ của mỗi đội

Mục đích: Phân tích độ "quyết liệt" hoặc kỷ luật thi đấu chung của toàn bộ giải đấu theo từng mùa.

```
select a.Team, avg(a.red*1.0), avg(yellow*1.0)
    from(
        select a.Hometeam Team, a.HomeRedCards red,a.HomeYellowCards yellow
        from epl_match_24 a
        union all
        select a.AwayTeam Team, a.AwayRedCards red,a.AwayYellowCards yellow
        from epl_match_24 a) a
group by a.Team
```

4) Tỷ lệ thắng trên sân nhà và sân khách của các đội

Mục đích: So sánh Lợi thế sân nhà/sân khách bằng cách phân tích tỷ lệ thắng

```

---7. Tỷ lệ thắng sân nhà và sân khách của từng đội
---Mục đích: Phân tích phong độ sân nhà/sân khách.
with
homestats as(
select a.HomeTeam Team, (count(case when a.FullTimeResult = 'H' then 1 end)*1.0 / count(a.FullTimeResult)) as per
from epl_match_24 a
group by a.HomeTeam),
awaystats as(
select a.AwayTeam Team, (count(case when a.FullTimeResult = 'A' then 1 end)*1.0 / count(a.FullTimeResult)) as per
from epl_match_24 a
group by a.AwayTeam)
select h.Team, format((h.per),'P2') as home, format((a.per),'P2') as away
from homestats h join awaystats a on h.Team = a.Team

```

5)Đội có trận hòa nhiều nhất

Mục đích: Xác định đội có Tính ổn định (an toàn) cao nhất thông qua số trận hòa

```

---9. Đội có số trận hòa nhiều nhất trong mùa
---Mục đích: Xác định đội "an toàn" nhất (ít thắng, ít thua)
select a.Season, b.Team, a.draw
from(
    select a.Season, max(a.draw) draw
    from
        (select a.Season, a.Team, sum(a.draw) draw
        from (
            select a.Season, a.HomeTeam Team, sum(case when a.FullTimeResult = 'D' then 1 else 0 end) draw
            from epl_match_24 a
            group by a.Season, a.HomeTeam
            union all
            select a.Season, a.AwayTeam Team, sum(case when a.FullTimeResult = 'D' then 1 else 0 end) draw
            from epl_match_24 a
            group by a.Season, a.AwayTeam) a
        group by a.Season, a.Team) a
    group by a.Season) a inner join (select a.Season, a.Team, sum(a.draw) draw
    from (
        select a.Season, a.HomeTeam Team, sum(case when a.FullTimeResult = 'D' then 1 else 0 end) draw
        from epl_match_24 a
        group by a.Season, a.HomeTeam
        union all
        select a.Season, a.AwayTeam Team, sum(case when a.FullTimeResult = 'D' then 1 else 0 end) draw
        from epl_match_24 a
        group by a.Season, a.AwayTeam) a
    group by a.Season, a.Team) b on a.Season = b.Season and a.draw = b.draw
    order by a.draw asc

```

CHƯƠNG 5 : PHÂN TÍCH BẢNG PYTHON

5.1 Phân tích bằng Python :

1) Tổng số điểm

Mục đích: Xác định đội mạnh nhất (thay vì chỉ nhìn bàn thắng) bằng cách áp dụng quy tắc tính điểm bóng đá (Thắng=3, Hòa=1, Thua=0)

-Dưới đây là câu truy vấn python dành cho mục đích trên:

```
df['homescore'] = df['FullTimeResult'].map({'H':3, 'D':1, 'A':0})
df['awayscore'] = df['FullTimeResult'].map({'H':0, 'D':1, 'A':3})
homescore = df.groupby(['HomeTeam', 'Season'])['homescore'].sum()
awayscore = df.groupby(['AwayTeam', 'Season'])['awayscore'].sum()
Total = homescore.add(awayscore, fill_value=0).reset_index()
Total.columns = ['Team', 'Season', 'TotalPoints']
display(Total.sort_values(['TotalPoints', 'Season'], ascending=[True, False]))
```

2) Hiệu suất sút trúng đích của đội chủ nhà và đội khách

Mục đích: Đánh giá Hiệu quả dứt điểm thông qua tỷ lệ sút trúng đích của từng đội.

-Dưới đây là câu truy vấn python dành cho mục đích trên:

```
homehit = df.groupby('Season')['HomeShotsOnTarget'].sum()/df.groupby('Season')['HomeShots'].sum()
awayhit = df.groupby('Season')['AwayShotsOnTarget'].sum()/df.groupby('Season')['AwayShots'].sum()
Total = pd.DataFrame({'HomeHit':homehit, 'AwayHit':awayhit})
display(Total)
```

3) Tỷ lệ thẻ vàng và thẻ đỏ của mỗi đội

Mục đích: Phân tích độ "quyết liệt" hoặc kỷ luật thi đấu chung của toàn bộ giải đấu theo từng mùa

-Dưới đây là câu truy vấn python dành cho mục đích trên:

```

YellowCard = pd.concat([
    df[['HomeTeam', 'HomeYellowCards']].rename(columns={'HomeTeam': 'Team', 'HomeYellowCards': 'YellowCards'}),
    df[['AwayTeam', 'AwayYellowCards']].rename(columns={'AwayTeam': 'Team', 'AwayYellowCards': 'YellowCards'})
])
RedCard = pd.concat([
    df[['HomeTeam', 'HomeRedCards']].rename(columns={'HomeTeam': 'Team', 'HomeRedCards': 'RedCards'}),
    df[['AwayTeam', 'AwayRedCards']].rename(columns={'AwayTeam': 'Team', 'AwayRedCards': 'RedCards'})
])
TotalY = YellowCard.groupby('Team')['YellowCards'].sum()
TotalMatches = YellowCard.groupby('Team')['YellowCards'].count()
AverageYellowCards = (TotalY / TotalMatches).rename('AverageYellowCards')
TotalR = RedCard.groupby('Team')['RedCards'].sum()
AverageRedCards = (TotalR / TotalMatches).rename('AverageRedCards')
Total = pd.merge(
    AverageYellowCards.reset_index(),
    AverageRedCards.reset_index(),
    on='Team'
)
display(Total.sort_values(['AverageYellowCards', 'AverageRedCards'], ascending=[True, False]))

```

4) Tỷ lệ thắng trên sân nhà và sân khách của các đội

Mục đích: So sánh Lợi thế sân nhà/sân khách bằng cách phân tích tỷ lệ thắng

-Dưới đây là câu truy vấn python dành cho mục đích trên:

```

df['HomeWin'] = df['FullTimeResult'].map({'H':1, 'D':0, 'A':0})
df['AwayWin'] = df['FullTimeResult'].map({'H':0, 'D':0, 'A':1})
HomeWin = df.groupby('HomeTeam')['HomeWin'].mean()
AwayWin = df.groupby('AwayTeam')['AwayWin'].mean()
Total = pd.DataFrame({'HomeWin':HomeWin, 'AwayWin':AwayWin}).reset_index()
Total.columns = ['Team', 'Home', 'Away']
display(Total)

```

5) Đội có trận hòa nhiều nhất

Mục đích: Xác định đội có Tính ổn định (an toàn) cao nhất thông qua số trận hòa

-Dưới đây là câu truy vấn python dành cho mục đích trên:

```

df['homedraw'] = df['FullTimeResult'].map({'H':0, 'D':1, 'A':0})
df['awaydraw'] = df['FullTimeResult'].map({'H':0, 'D':1, 'A':0})
HomeDraw = df.groupby(['HomeTeam', 'Season'])['homedraw'].sum()
AwayDraw = df.groupby(['AwayTeam', 'Season'])['awaydraw'].sum()
Total = HomeDraw.add(AwayDraw, fill_value=0).reset_index()
Total.columns = ['Team', 'Season', 'TotalDraws']
MaxDraw = Total.groupby('Season')['TotalDraws'].max().reset_index()
MaxDraw.columns = ['Season', 'MaxDraws']
Result = Total.merge(MaxDraw, on='Season')
Result = Result[Result['TotalDraws'] == Result['MaxDraws']]
display(Result)

```

5.1.2 KẾT QUẢ PHÂN TÍCH CỦA CODE SQL VÀ PYTHON

1) Tổng số điểm

	Team	Season	TotalPoints
37	West Ham	2024/25	37
35	Tottenham	2024/25	38
16	Everton	2024/25	39
27	Man United	2024/25	39
21	Liverpool	2023/24	40
39	Wolves	2024/25	41
14	Crystal Palace	2024/25	46
0	Arsenal	2023/24	49
18	Fulham	2024/25	51
24	Man City	2023/24	51
7	Brentford	2024/25	52
9	Brighton	2024/25	52
5	Bournemouth	2024/25	53
3	Aston Villa	2024/25	60
31	Nott'm Forest	2024/25	61
12	Chelsea	2024/25	63
29	Newcastle	2024/25	63
25	Man City	2024/25	64
1	Arsenal	2024/25	67
22	Liverpool	2024/25	82
32	Sheffield United	2023/24	7
33	Southampton	2024/25	11
23	Luton	2023/24	11
10	Burnley	2023/24	13
30	Nott'm Forest	2023/24	16
8	Brighton	2023/24	18
38	Wolves	2023/24	18
36	West Ham	2023/24	19
6	Brentford	2023/24	20
20	Leicester	2024/25	21
19	Ipswich	2024/25	22
15	Everton	2023/24	22
4	Bournemouth	2023/24	23
17	Fulham	2023/24	23
2	Aston Villa	2023/24	26
34	Tottenham	2023/24	27
13	Crystal Palace	2023/24	28
26	Man United	2023/24	29
28	Newcastle	2023/24	31
11	Chelsea	2023/24	35

2) Hiệu suất sút trúng đích của đội chủ nhà và đội khách

	HomeHit	AwayHit
Season		
2023/24	0.356396	0.375326
2024/25	0.351452	0.349788

3) Tỷ lệ thẻ vàng và thẻ đỏ của mỗi đội

	Team	AverageYellowCards	AverageRedCards
14	Man City	1.370370	0.018519
16	Newcastle	1.685185	0.037037
0	Arsenal	1.735849	0.094340
12	Liverpool	1.740741	0.055556
5	Burnley	1.833333	0.166667
3	Brentford	1.833333	0.037037
13	Luton	1.842105	0.000000
20	Tottenham	1.886792	0.018868
18	Sheffield United	1.888889	0.166667
21	West Ham	1.888889	0.092593
7	Crystal Palace	1.981132	0.094340
1	Aston Villa	1.981132	0.056604
4	Brighton	2.000000	0.074074
9	Fulham	2.037736	0.056604
15	Man United	2.037736	0.056604
17	Nott'm Forest	2.037736	0.037736
8	Everton	2.094340	0.037736
22	Wolves	2.207547	0.056604
10	Ipswich	2.314286	0.142857
11	Leicester	2.314286	0.000000
19	Southampton	2.457143	0.085714
2	Bournemouth	2.481481	0.074074
6	Chelsea	2.584906	0.037736

4) Tỷ lệ thắng trên sân nhà và sân khách của các đội

	Team	Home	Away
0	Arsenal	0.666667	0.615385
1	Aston Villa	0.481481	0.423077
2	Bournemouth	0.407407	0.333333
3	Brentford	0.407407	0.333333
4	Brighton	0.370370	0.259259
5	Burnley	0.111111	0.111111
6	Chelsea	0.666667	0.384615
7	Crystal Palace	0.407407	0.307692
8	Everton	0.333333	0.153846
9	Fulham	0.384615	0.370370
10	Ipswich	0.058824	0.166667
11	Leicester	0.166667	0.117647
12	Liverpool	0.814815	0.555556
13	Luton	0.222222	0.000000
14	Man City	0.714286	0.576923
15	Man United	0.384615	0.296296
16	Newcastle	0.576923	0.464286
17	Nott'm Forest	0.423077	0.407407
18	Sheffield United	0.000000	0.111111
19	Southampton	0.058824	0.055556
20	Tottenham	0.461538	0.259259
21	West Ham	0.250000	0.230769
22	Wolves	0.346154	0.296296

5) Đội có trận hòa nhiều nhất

	Team	Season	TotalDraws	MaxDraws
10	Burnley	2023/24	7	7
15	Everton	2023/24	7	7
16	Everton	2024/25	15	15
36	West Ham	2023/24	7	7

5.2 : Phân tích dữ liệu và thống kê mô tả bằng Python

Ở bước này, dữ liệu đã được đọc và phân tích cơ bản bằng thư viện Pandas trong Python nhằm giúp hiểu rõ hơn về đặc điểm tổng thể của tập dữ liệu trước khi xây dựng mô hình.

Cụ thể, lệnh `df.describe()` được sử dụng để tính toán các chỉ số thống kê mô tả cho các biến số trong tập dữ liệu `matches_cleaned.csv` (File đã được xử lý ở phần 3).

Các chỉ số bao gồm:

- count –(số lượng giá trị hợp lệ trong mỗi cột)
- mean – (giá trị trung bình)
- std – (độ lệch chuẩn)
- min / max –(giá trị nhỏ nhất và lớn nhất)
- 25%, 50%, 75% (các mốc phân vị (quartile) mô tả phân bố dữ liệu)

Dưới đây là bảng thống kê kết quả được tạo ra từ câu lệnh `df.describe()`:

Thống kê	Goals	Shots	Shots on Target (SOT)	Corners	Fouls	Yellow Cards	Red Cards

Đội nhà (Home)							
Count	534	534	534	534	534	534	534
Mean	1.65	14.57	5.15	5.71	10.69	1.86	0.06
Std Dev	1.34	6.09	2.73	3.29	3.60	1.36	0.25
Min	0	1	0	0	2	0	0
Median (50%)	1	14	5	5	11	2	0
Max	7	36	16	17	21	7	2
Đội khách (Away)							
Count	534	534	534	534	534	534	534
Mean	1.49	12.28	4.40	4.89	11.32	2.15	0.06

Std Dev	1.25	5.50	2.42	3.00	3.63	1.38	0.24
Min	0	1	0	0	1	0	0
Median (50%)	1	12	4	4	11	2	0
Max	6	37	15	18	22	8	1

(Bảng 5.2: Thống kê mô tả của các biến số trong tập dữ liệu)

Nhận xét tổng quan:

- Trung bình mỗi trận, đội chủ nhà ghi 1.65 bàn, trong khi đội khách ghi 1.49 bàn, cho thấy lợi thế sân nhà có ảnh hưởng nhẹ đến kết quả.
- Số cú sút trung bình của đội chủ nhà (≈ 14.6) cao hơn đội khách (≈ 12.3), đồng thời số cú sút trúng đích (SOT) cũng cao hơn (5.15 so với 4.40).
- Trung bình, mỗi đội thực hiện khoảng 5–6 quả phạt góc mỗi trận. Đội chủ nhà vẫn nhỉnh hơn một chút so với đội khách.
- Thống kê thẻ phạt cho thấy đội khách thường nhận nhiều thẻ vàng hơn (2.15 so với 1.86), trong khi thẻ đỏ khá hiếm (trung bình ~ 0.06 cho cả hai bên).
- Các giá trị độ lệch chuẩn (std) cho thấy sự phân tán khá lớn ở số cú sút và bàn thắng, phản ánh sự khác biệt giữa các đội mạnh và yếu.

Tổng kết phần 5.2

Phần này giúp nhóm hiểu rõ đặc điểm cơ bản của tập dữ liệu, bao gồm xu hướng ghi bàn, số cú sút, phạt góc và mức độ thẻ phạt.

Kết quả này làm nền cho các bước tiếp theo — đặc biệt là việc **xây dựng mô hình hồi quy dự đoán tỷ số** ở phần 7.

CHƯƠNG 6 : TRỰC QUAN HÓA

6.1 Trực quan hóa dữ liệu

1 Tổng số điểm của mỗi đội trong mùa giải

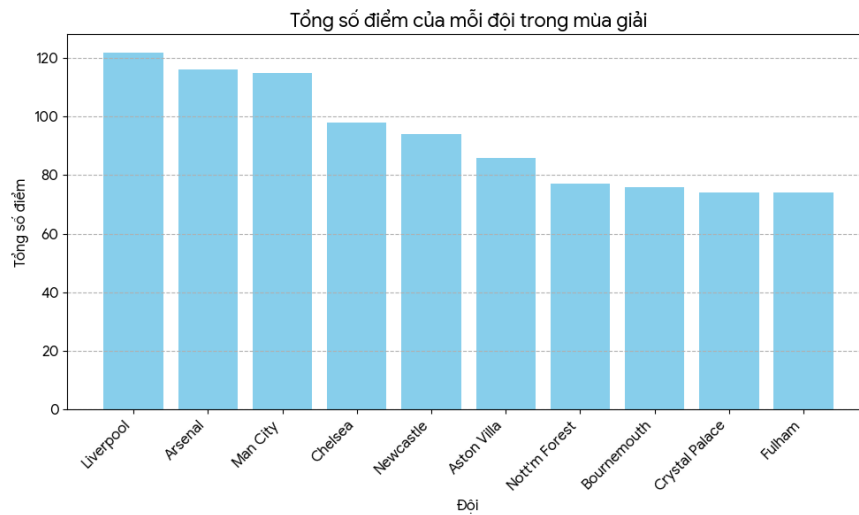
- Mục đích: Xác định đội mạnh nhất (thay vì chỉ nhìn bàn thắng) (biểu đồ cột)

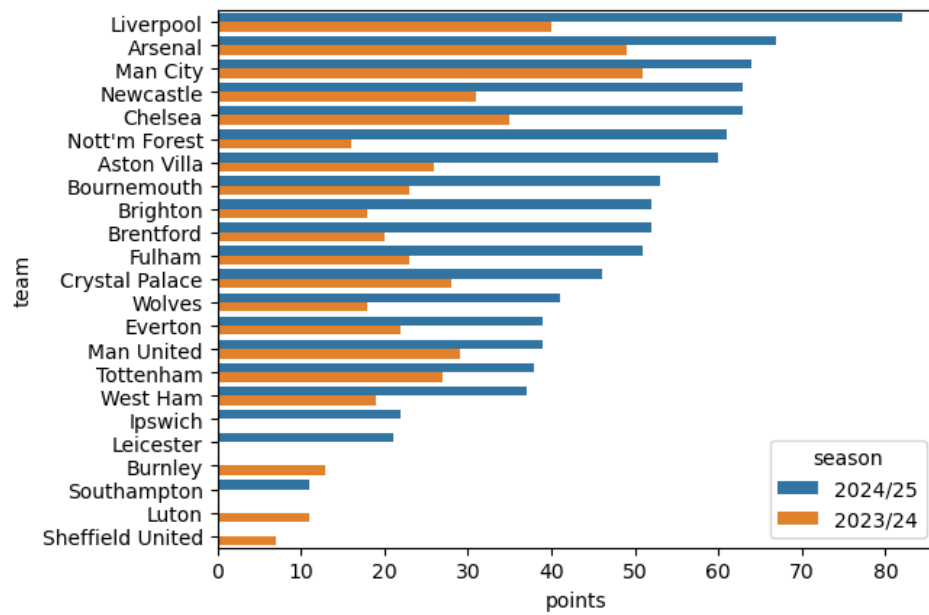
- Mùa giải: Lấy 2 mùa 2023/2024 và 2024/2025

- Cách tính điểm:

+ Trận thắng x3 điểm

+ Trận hoà x1 điểm



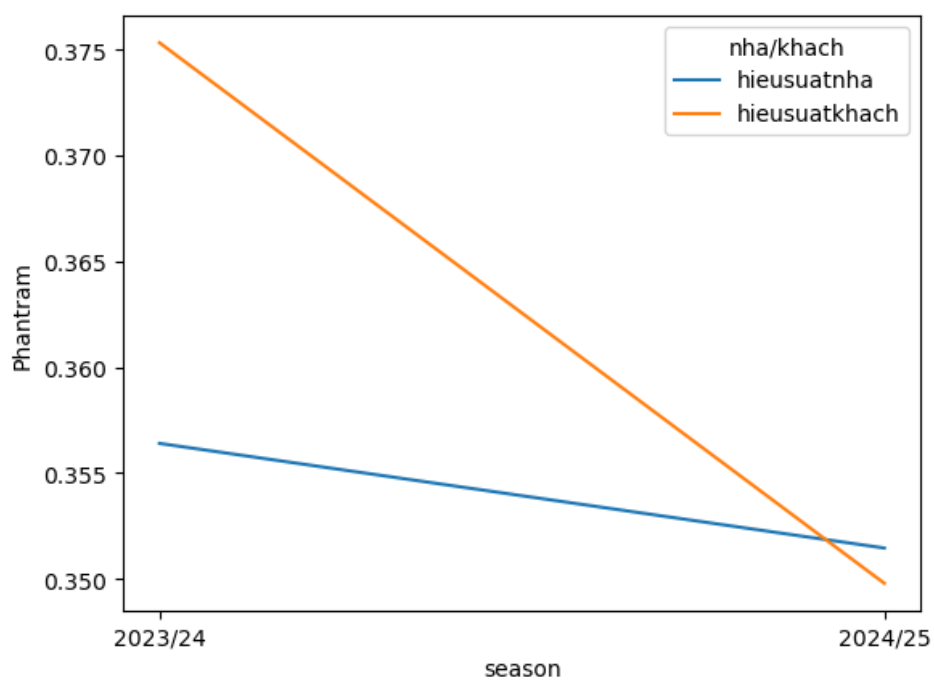
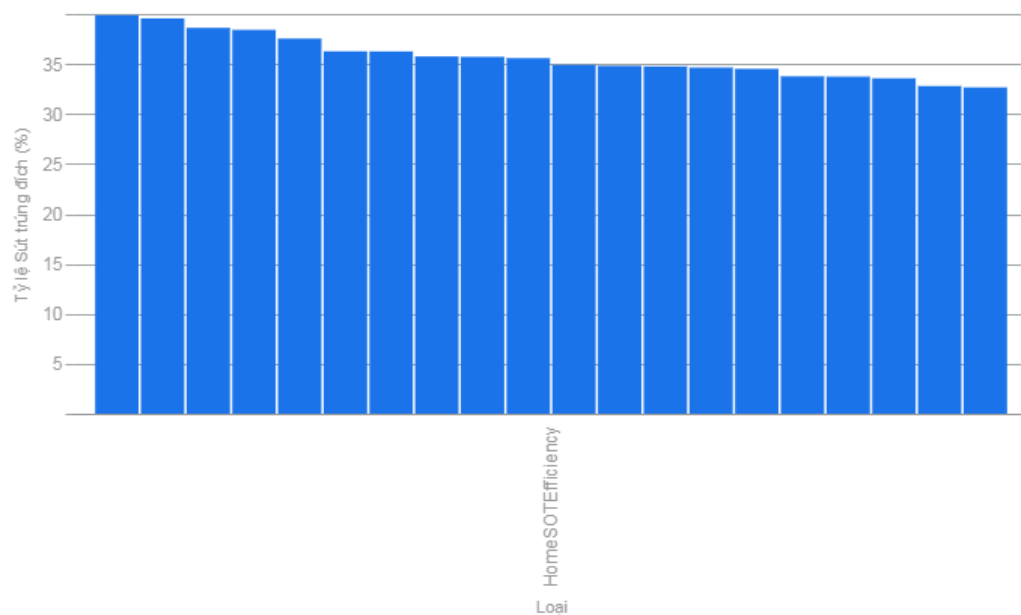


2 Hiệu suất sút trúng đích của đội chủ nhà và đội khách

- Mục đích: Đánh giá hiệu quả dứt điểm

- Cách tính: Tổng cú sút trúng đích / Tổng cú sút thực hiện

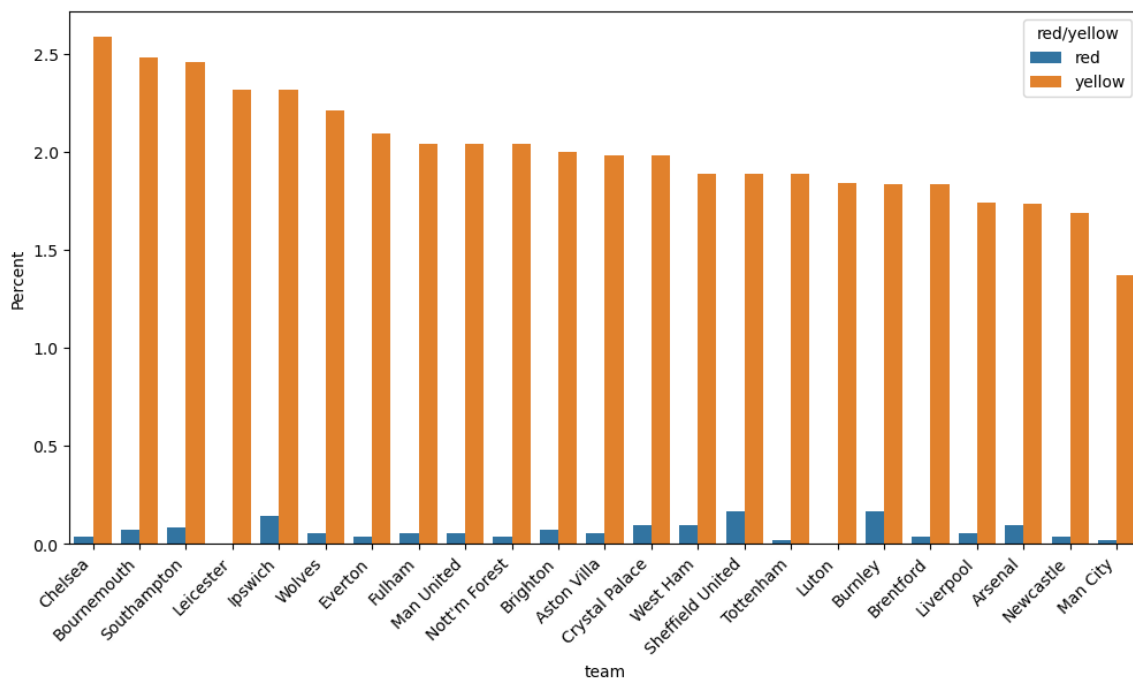
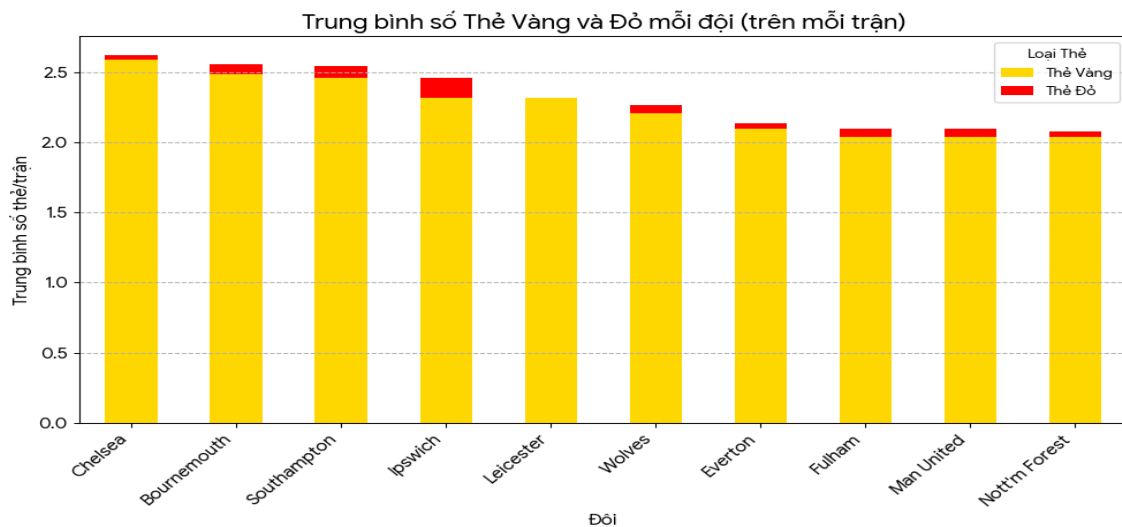
Hiệu suất Sút trúng đích (SOT) của đội Chủ nhà và đội Khách



3 Trung bình số thẻ vàng và đỏ mỗi đội

- Mục đích: Đánh giá tính kỷ luật của đội.

- Cách tính: Tổng thẻ đỏ thẻ vàng / số trận thi đấu



4 Tỷ lệ thắng sân nhà và sân khách của từng đội

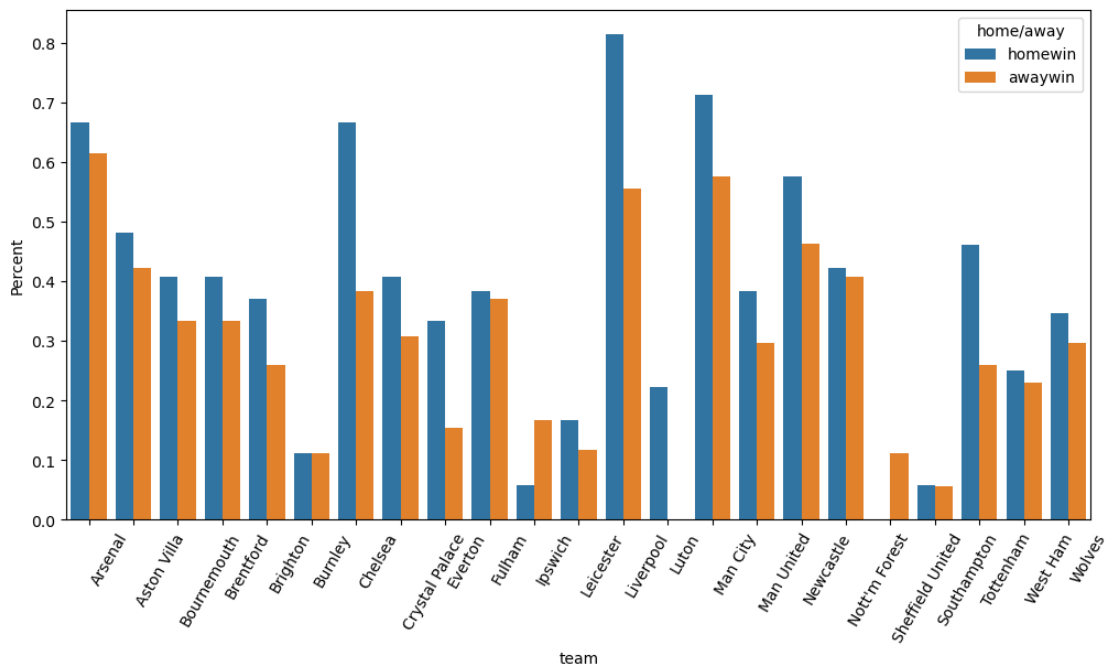
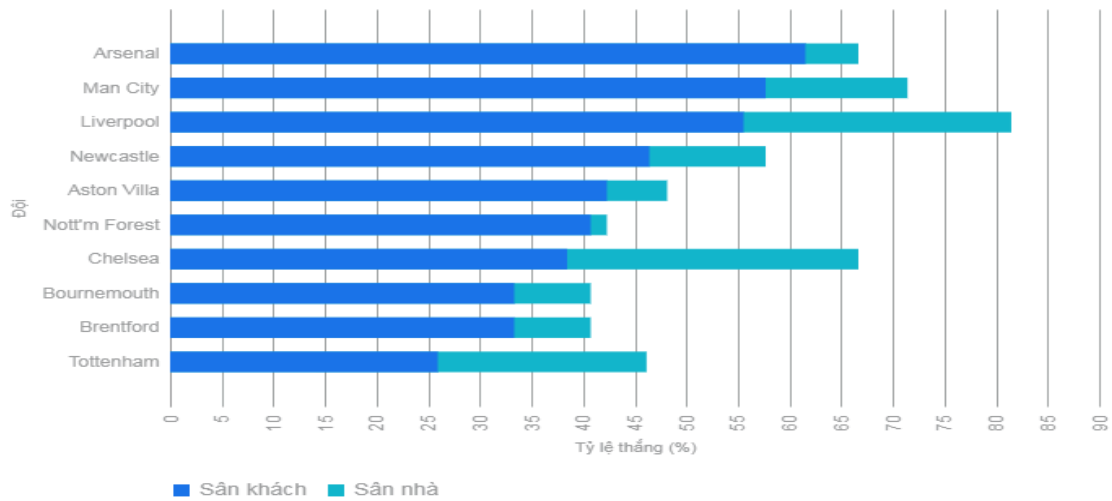
- Mục đích: Phân tích phong độ sân nhà/sân khách.

- Cách tính:

+ Sân nhà: Tổng các trận thắng ở sân nhà / Tổng các trận ở sân nhà

+ Sân khách: Tổng các trận thắng ở sân khách / Tổng các trận ở sân khách

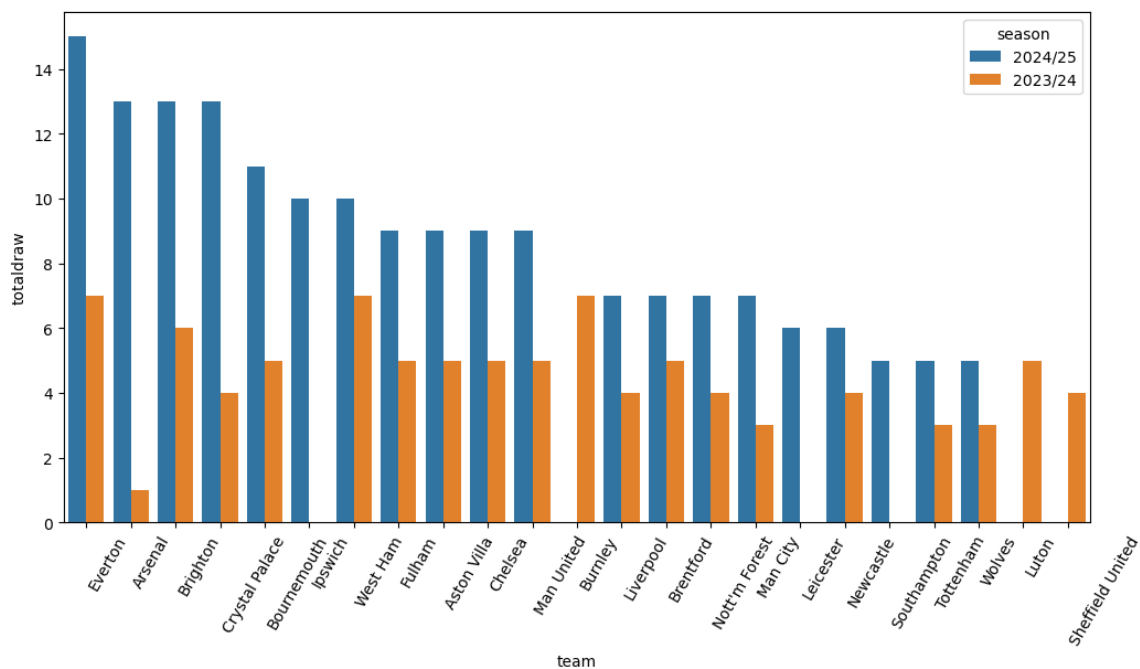
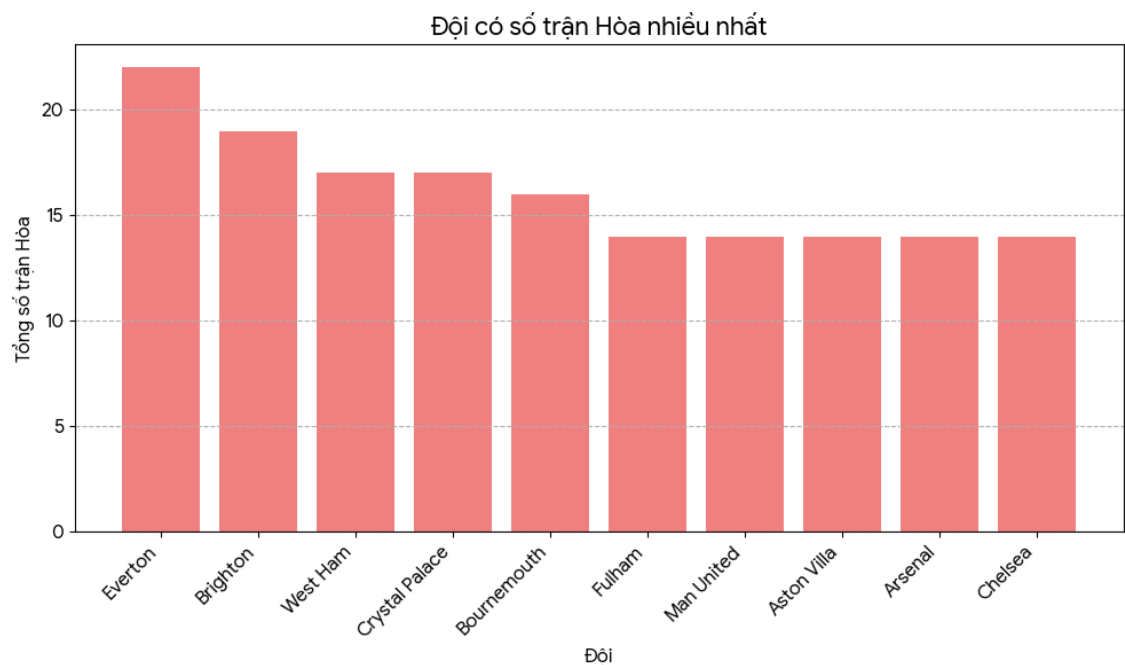
Tỷ lệ Thắng Sân nhà và Sân khách của mỗi đội (Top 10)



5 Đội có số trận hòa nhiều nhất trong mùa

- Mục đích: Xác định đội "an toàn" nhất (ít thắng, ít thua).

- Cách tính: Tổng số trận hoà ở sân nhà + Tổng số trận hoà ở sân khách



6.2. Kết luận (Dựa trên kết quả phân tích)

1. **Về sức mạnh:** **Liverpool** là đội mạnh nhất về điểm số tổng thể, trong khi **Arsenal** thể hiện phong độ sân khách ổn định nhất.
2. **Về hiệu suất:** **Brentford** là đội có hiệu suất sút trúng đích cao nhất, chứng tỏ độ chính xác cao trong các pha dứt điểm.
3. **Về kỷ luật:** **Chelsea** là đội có vấn đề kỷ luật rõ rệt nhất với số lượng thẻ vàng trung bình trên mỗi trận đấu cao nhất.
4. **Về tính an toàn:** **Everton** là đội có số trận hòa nhiều nhất, cho thấy họ thường là đội khó bị đánh bại nhưng cũng khó giành trọn 3 điểm.

6.3. Hạn chế

- Phân tích chỉ tập trung vào các chỉ số đầu ra (Output metrics) mà chưa đi sâu vào các chỉ số chiến thuật (Input metrics) như Tỷ lệ kiểm soát bóng, Số lần cắt bóng, v.v.
- Dữ liệu tổng hợp từ hai mùa giải, có thể làm mờ sự thay đổi phong độ của các đội qua các giai đoạn.

6.4. Đề xuất

- Tiếp tục phân tích các chỉ số sâu hơn như **Hiệu số bàn thắng bại** và **Tỷ lệ chuyển hóa cơ hội** để có cái nhìn toàn diện hơn.
- Phân tích dữ liệu theo từng mùa giải riêng biệt để đánh giá xu hướng phát triển phong độ qua thời gian.

CHƯƠNG 7 : PHÂN TÍCH HỒI QUY

7.1 Feature Engineering and Data Preparation

Trong phần này, tập dữ liệu được chuẩn bị bằng cách loại bỏ các cột không liên quan hoặc có khả năng gây sai lệch trước khi xây dựng mô hình hồi quy..

7.1.1 Data Preparation

Cơ sở lý luận:

những features hiện tại như full-time results và half-time results đã tiết lộ thông tin sau trận đấu và có thể dẫn đến rò rỉ dữ liệu nếu được đưa vào mô hình.

- **Hành động:**

Loại bỏ những cột chứa dữ liệu quan trọng đến kết quả (ft_result, ht_result, ht_home_goals, ht_away_goals).

Chỉ giữ lại các thống kê trận đấu có thể quan sát được và thông tin có sẵn trước trận đấu.

- **Mục tiêu :**

Đảm bảo rằng tất cả các đặc trưng còn lại chỉ đại diện cho thông tin có sẵn trước khi **trận đấu bắt đầu**.

Column Name	Reason for Removal
ft_result	Tiết lộ kết quả trận đấu (data leakage)
ht_result	Tiết lộ kết quả hiệp một
ht_home_goals	Chứa thông tin về hiệp 1
ht_away_goals	Chứa thông tin về hiệp 1

Table 7.1.1: Columns removed prior to modeling

Đoạn mã Python dưới đây minh họa cách các cột không cần thiết được loại bỏ trước khi huấn luyện mô hình.

```
#Xóa những cột có thể làm lộ thông tin
drop_cols = ["ft_result", "ht_result", "ht_home_goals", "ht_away_goals"]

df_model = df.drop(columns=[col for col in drop_cols if col in df.columns])

# Xếp dữ liệu theo thời gian trận đấu
df_model = df_model.sort_values(by="match_date").reset_index(drop=True)
```

Figure 7.1.1: Python code for column selection and preparation.

- Sau khi loại bỏ các đặc trưng không liên quan và liên quan đến kết quả, tập dữ liệu chỉ còn lại các chỉ báo trước trận đấu và các biến hiệu suất có thể quan sát được.
- Các thuộc tính này sau đó được sử dụng để thiết kế các đặc trưng thống kê mới, chẳng hạn như phong độ gần đây và bàn thắng trung bình, được thảo luận trong Section 7.1.2 Feature Engineering

7.1.2 Feature Engineering

- Trong bước này, các Feature mới được tạo ra để đại diện tốt hơn cho phong độ gần đây của mỗi đội..
- Thay vì chỉ nhìn chỉ số của 1 trận đấu, Mô hình phải hiểu được phong độ tổng thể của 1 đội — liệu họ đang thi đấu tốt hay không trong các trận gần đây.
- Để làm được việc đó, chúng tôi đã tính toán average number of goals, shots on target, corners của 5 trận trước đó cho cả home và away teams.
- Điều này giúp mô hình nắm bắt được các xu hướng ngắn hạn có thể ảnh hưởng đến kết quả trận đấu.

Đoạn Python code dưới đây cho thấy cách tính toán các chỉ số trung bình cho từng đội .

```

import pandas as pd
df = pd.read_csv('matches_cleaned.csv')
# tạo hàm tính phong độ trung bình 5 trận gần nhất cho mỗi đội
def tinh_phong_do(df, team_prefix):
    """
    Hàm này tính phong độ gần đây cho từng đội
    bao gồm trung bình bàn thắng, sút trúng đích và phạt góc
    trong 5 trận gần nhất.
    """
    df_team = df.copy()

    def calculate_rolling_feature(series):
        # Tính rolling mean trên cửa sổ 5 trận, tối thiểu 1 trận
        rolling_mean = series.rolling(window=5, min_periods=1).mean()
        # Dịch chuyển (shift) kết quả 1 vị trí để lấy giá trị TRƯỚC trận hiện tại
        return rolling_mean.shift(1)

    # Trung bình bàn thắng 5 trận gần nhất
    df_team[f'{team_prefix}_avg_goals_last5'] = (
        df_team.groupby(f'{team_prefix}_team')[f'{team_prefix}_goals']
        .transform(lambda x: x.rolling(window=5, min_periods=1).mean())
    )
    # Trung bình sút trúng đích 5 trận gần nhất
    df_team[f'{team_prefix}_avg_sot_last5'] = (
        df_team.groupby(f'{team_prefix}_team')[f'{team_prefix}_sot']
        .transform(lambda x: x.rolling(window=5, min_periods=1).mean())
    )
    # Trung bình phạt góc 5 trận gần nhất
    df_team[f'{team_prefix}_avg_corners_last5'] = (
        df_team.groupby(f'{team_prefix}_team')[f'{team_prefix}_corners']
        .transform(lambda x: x.rolling(window=5, min_periods=1).mean())
    )
    return df_team

```

```

# 4 Áp dụng hàm cho đội nhà và đội khách
df = tinh_phong_do(df, team_prefix: "home")
df = tinh_phong_do(df, team_prefix: "away")

# 5 Kiểm tra các cột đặc trưng mới được tạo
print([col for col in df.columns if 'last5' in col])

# 6 Lưu dữ liệu đã tạo feature để sử dụng cho mô hình hồi quy
df.to_csv("team_stats_features.csv", index=False)

```

Figure 7.1.2: Python code for creating 5-match average performance features.

Dưới đây là các features mới được tạo ra từng đoạn code ở trên và mô tả của chúng .

Feature Name	Description
home_avg_goals_last5	Số bàn thắng trung bình được ghi bởi đội nhà trong 5 trận gần nhất
away_avg_goals_last5	Số bàn thắng trung bình được ghi bởi đội khách trong 5 trận gần nhất
home_avg_sot_last5	Cú sút trúng đích trung bình của đội nhà (5 trận gần nhất)
away_avg_sot_last5	Cú sút trúng đích trung bình của đội khách (5 trận gần nhất)
home_avg_corners_last5	Phạt góc trung bình của đội nhà (5 trận gần nhất)
away_avg_corners_last5	Phạt góc trung bình của đội khách (5 trận gần nhất)

Table 7.1.2: New features created from recent team performance

Sau khi tạo các đặc trưng này, tập dữ liệu hiện bao gồm cả thông kê trận đấu gốc và các chỉ báo mới đại diện cho phong độ gần đây của mỗi đội.

Các đặc trưng được cập nhật này sẽ được sử dụng trong bước tiếp theo, nơi tên đội được mã hóa và dữ liệu được chuẩn hóa tỷ lệ trước khi huấn luyện mô hình hồi quy, như được mô tả trong Section 7.1.3 Data Encoding and Scaling.

7.1.3 Data Encoding and Scaling

Trước khi huấn luyện mô hình hồi quy, bắt buộc phải xử lý dữ liệu thô để các thuật toán học máy có thể hiểu và xử lý chúng một cách hiệu quả. Chúng tôi tập trung vào hai bước chính: chuyển đổi dữ liệu phân loại và chuẩn hóa các đặc trưng số.

1. Categorical Feature Encoding (Mã hóa Đặc trưng Phân loại)

- Vấn đề: Tên đội (home_team, away_team) là dữ liệu dạng văn bản mà mô hình hồi quy không thể xử lý trực tiếp.
- Giải pháp: Chúng tôi áp dụng Mã hóa Nhãn (Label Encoding) để gán một giá trị số nguyên duy nhất cho mỗi đội. Ví dụ: "Manchester City" có thể được gán 1,

"Liverpool" 2, v.v. Mặc dù cách tiếp cận đơn giản này có thể ngầm tạo ra một mối quan hệ thứ tự (ordinal) giữa các đội, nó là một giải pháp trực tiếp và nhanh chóng cho trường hợp này.

2. Numerical Feature Scaling(Chuẩn hóa Đặc trưng Số)

Vấn đề: Các đặc trưng số có phạm vi giá trị khác nhau rất lớn. Ví dụ, tổng số cú sút có thể lên đến vài chục, trong khi số phạt góc thường chỉ là các chữ số đơn. Nếu không chuẩn hóa, các đặc trưng có giá trị lớn sẽ chi phối và làm sai lệch mô hình trong quá trình học.

Giải pháp : Áp dụng Standard Scaling (Z-Score Normalization). Phương pháp này biến đổi mọi giá trị x thành giá trị chuẩn hóa z bằng công thức $z = \frac{x-\mu}{\sigma}$ với μ là mu giá trị trung bình và σ là độ lệch chuẩn của các feature đó .

Kết quả là tất cả các đặc trưng số sẽ có mean bằng 0 và sigma bằng 1, đảm bảo chúng đóng góp công bằng vào quá trình huấn luyện mô hình.

Đoạn mã Python dưới đây cho thấy cách tên đội được mã hóa và các đặc trưng được chuẩn hóa tỷ lệ để huấn luyện mô hình.

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

# ❶ Đọc file dữ liệu sau khi tạo feature
df = pd.read_csv("team_stats_features.csv")

# ❷ mã hóa tên đội (Categorical Encoding)
encoder = LabelEncoder()
# Mã hóa tên đội cho cả đội nhà và đội khách
df["home_team_encoded"] = encoder.fit_transform(df["home_team"])
df["away_team_encoded"] = encoder.fit_transform(df["away_team"])

# ❸ CHỌN CÁC CỘT FEATURE CẦN CHUẨN HÓA (Numerical Scaling)
# Các đặc trưng thống kê trong 5 trận gần nhất (last5) và các chỉ số trận đấu (shots, corners, fouls)
num_features = [col for col in df.columns if
                 col.endswith("last5") or
                 "shots" in col or
                 "corners" in col or
                 "fouls" in col]

# ❹ KHỞI TẠO VÀ ÁP DỤNG StandardScaler
scaler = StandardScaler()
# Chuẩn hóa các cột đặc trưng số
df[num_features] = scaler.fit_transform(df[num_features])

# ❺ Lưu dataset đã mã hóa và chuẩn hóa
df.to_csv( path_or_buf: "matches_encoded_scaled.csv", index=False)

# ❻ Kiểm tra nhanh các cột mới
print(df.head(3)[["home_team", "home_team_encoded", "away_team", "away_team_encoded"]])

```

Figure 7.1.3: Python code for encoding team names and scaling numerical features.

Step	Description	Method Used
Encoding	Chuyển đổi tên đội sang định dạng số	LabelEncoder
Scaling	Chuẩn hóa các đặc trưng số sang phạm vi chung	StandardScaler
Output File	Tập dữ liệu cuối cùng để huấn luyện mô hình	matches_encoded_scaled.csv

Table 7.1.3: Data preprocessing summary

3.summary

- Sau bước này, tập dữ liệu hoàn toàn là số và sẵn sàng để huấn luyện mô hình.
- Tất cả các đặc trưng hiện đều chia sẻ một tỷ lệ nhất quán, ngăn chặn các giá trị lớn hơn chi phối quá trình học.

- Dữ liệu đã được xử lý được lưu dưới dạng `matches_encoded_scaled.csv` và sẽ được sử dụng trong phần kế tiếp để xây dựng mô hình hồi quy là 7.2 Model Training and Evaluation.

7.2 Model Training and Evaluation (Huấn luyện và Đánh giá Mô hình)

Phần này mô tả việc triển khai Hồi quy Poisson (Poisson Regression, một phương pháp thống kê được chọn vì tính phù hợp vượt trội với dữ liệu đếm (count data) như số bàn thắng. Chúng tôi đã áp dụng cách tiếp cận mô hình kép (dual-model approach để huấn luyện các mô hình riêng biệt cho dự đoán bàn thắng đội nhà và đội khách.

7.2.1 Feature Selection and Data Consistency (Lựa chọn Đặc trưng và Tính nhất quán Dữ liệu)

Để đảm bảo mô hình Poisson hoạt động tối ưu, chúng tôi đã sử dụng tập dữ liệu `matches_encoded_scaled.csv` đã được chuẩn hóa.

- Tính phù hợp: Không giống như các mô hình dựa trên cây, các mô hình hồi quy tuyến tính tổng quát (GLM) như Poisson Regression hoạt động hiệu quả hơn với các đặc trưng số đã được chuẩn hóa (Standard Scaling).
- Đặc trưng quan trọng: Tập hợp các đặc trưng đầu vào (features) được mở rộng để bao gồm các cột đã mã hóa (`home_team_encoded` và `away_team_encoded` cùng với các chỉ số thống kê hiệu suất gần đây (avg, shots, corners).
- Tính nhất quán: Dữ liệu được chia chỉ một lần bằng `train_test_split` trên các đặc trưng (X) và mục tiêu chính (`home_goals`) để đảm bảo cả hai mô hình (Home Goals và Away Goals) được huấn luyện và kiểm tra trên cùng một tập hợp trận đấu.

7.2.2 Dual Model Training and Evaluation (Huấn luyện và Đánh giá Mô hình Kép)

Hai mô hình Poisson Regressor độc lập được khởi tạo với tham số (regularization) và được huấn luyện trên tập `X_train` nhất quán, dự đoán các biến mục tiêu tương ứng của chúng. Hiệu suất sau đó được đánh giá bằng các chỉ số hồi quy tiêu chuẩn: MAE, RMSE và R^2

Python Implementation (Triển khai Python)

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import PoissonRegressor # Thư viện Hồi quy Poisson
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1 Đọc file dữ liệu ĐÃ MÃ HÓA VÀ CHUẨN HÓA
df = pd.read_csv("matches_encoded_scaled.csv")

# 2 Chọn các cột feature đầu vào và mục tiêu
# BẮT BUỘC: Thêm các cột 'encoded' để mô hình biết đội nào đang thi đấu
features = [col for col in df.columns if
             'avg' in col or
             'shots' in col or
             'corners' in col or
             'encoded' in col]
target_home = 'home_goals'
target_away = 'away_goals'

# 3 Chia dữ liệu thành tập huấn luyện và kiểm tra (80/20) CHO CẢ HAI MỤC TIÊU
X_train, X_test, y_train_home, y_test_home = train_test_split(
    *arrays: df[features], df[target_home], test_size=0.2, random_state=42
)
y_train_away = df.loc[y_train_home.index, target_away]
y_test_away = df.loc[y_test_home.index, target_away]

```

```

# 4 Khởi tạo mô hình Poisson Regression
# Alpha là tham số regularization (L2), giữ giá trị nhỏ để tránh underfitting.
model_home = PoissonRegressor(alpha=1e-6, max_iter=500, tol=1e-3)
model_away = PoissonRegressor(alpha=1e-6, max_iter=500, tol=1e-3)

# 5 Huấn luyện mô hình cho đội nhà và đội khách (Sử dụng X_train chung)
print("Training Poisson Models...")
model_home.fit(X_train, y_train_home)
model_away.fit(X_train, y_train_away)

# 6 Dự đoán kết quả trên tập kiểm tra (Sử dụng X_test chung)
y_pred_home = model_home.predict(X_test)
y_pred_away = model_away.predict(X_test)

# 7 Tính toán các chỉ số đánh giá
mae_home = mean_absolute_error(y_test_home, y_pred_home)
rmse_home = np.sqrt(mean_squared_error(y_test_home, y_pred_home))
r2_home = r2_score(y_test_home, y_pred_home)

mae_away = mean_absolute_error(y_test_away, y_pred_away)
rmse_away = np.sqrt(mean_squared_error(y_test_away, y_pred_away))
r2_away = r2_score(y_test_away, y_pred_away)

# 8 In kết quả đánh giá (Bổ sung RMSE)
print("Poisson Home Goals Prediction Results:")
print(f"MAE: {mae_home:.3f}, RMSE: {rmse_home:.3f}, R²: {r2_home:.3f}")
print("Poisson Away Goals Prediction Results:")
print(f"MAE: {mae_away:.3f}, RMSE: {rmse_away:.3f}, R²: {r2_away:.3f}")

```

Figure 7.2.2: Python code for training and evaluating separate Poisson Regressor models for Home and Away Goals

7.3 Analysis of Results

Phần này trình bày đánh giá chi tiết về hiệu suất của mô hình Poisson Regression trên Tập kiểm tra (Test Set), được xác nhận là mô hình hiệu quả nhất cho bài toán dự đoán bàn thắng.

7.3.2 Numerical Evaluation

Các chỉ số hồi quy thu được sau khi sử dụng Poisson Regression với các đặc trưng đã mã hóa được tóm tắt trong Bảng 7.5:

Metric	Home Goals Model	Away Goals Model
MAE (Mean Absolute Error)	0.765 goals	0.713 goals
RMSE (Root Mean Squared Error)	1.022 goals	0.918 goals
R^2 (Coefficient of Determination)	0.355	0.429

Table 7.3.1 : Final Poisson Regression Model Evaluation Results

Tóm tắt: Chỉ số $R^2 = 0.429$ cho dự đoán bàn thắng đội khách là kết quả tốt nhất đạt được trong tất cả các mô hình đã thử nghiệm, cho thấy mô hình Poisson đã giải thích được gần 43% sự biến thiên của số bàn thắng thực tế. Độ sai lệch trung bình chỉ khoảng 0.7 bàn

7.3.2 Visual Analysis (Phân tích Trực quan)

Biểu đồ phân tán so sánh giá trị thực tế (Actual) và dự đoán (Predicted) của mô hình Poisson Regression minh họa rõ ràng cách mô hình xử lý dữ liệu đếm (Count data) và mối quan hệ giữa hai biến:

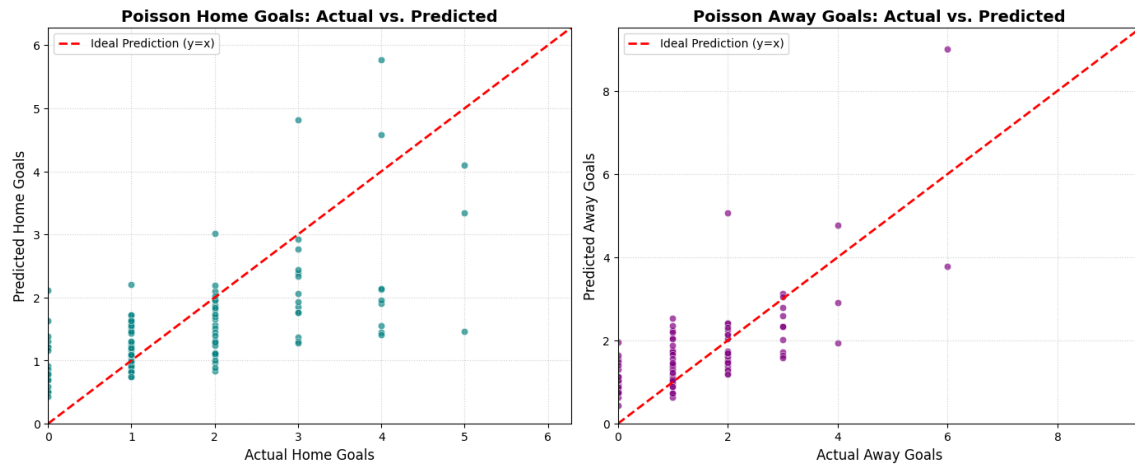


Figure 7.6: Comparison of Actual vs. Predicted Goals using Poisson Regression.

Diễn giải Biểu đồ

1. Tự Thích ứng với Phân phối (Distribution):

Phản ánh Thực tế:

Các điểm dữ liệu tập trung dày đặc ở khu vực 0 đến 2 bàn thắng, phản ánh bản chất ít bàn thắng và sự phân phối không đối xứng (skewed) của dữ liệu bóng đá.

Xử lý Giá trị 0 Bàn:

Mô hình Poisson thể hiện khả năng xử lý giá trị 0 bàn thắng hiệu quả, với các điểm dự đoán (trục Y) tập trung sát về phía trục X khi Actual = 0.

2. Độ Chính xác và Hạn chế (Focus on Ideal Line $y=x$):

Tập trung gần Đường Lý tưởng:

- Các điểm dữ liệu nằm gần đường đồ (đường y bằng x), đặc biệt cho các giá trị Actual từ 1 đến 3 bàn, chứng minh mô hình hoạt động hiệu quả trong việc dự đoán các kết quả phổ biến nhất.

Xu hướng Bảo thủ (Hạn chế Lớn nhất):

- Mô hình có xu hướng dự đoán quá thấp (underpredict) các sự kiện tỷ số cao. Các chấm nằm rõ ràng dưới đường đồ khi Actual Goals là 5 hoặc 6, cho thấy mô hình gặp khó khăn trong việc dự đoán các trận đấu có tính chất ngoại lai (high-scoring outliers).

3.Sự khác biệt Home vs. Away:

Hiệu suất Tách biệt:

- Biểu đồ Away Goals (Bên phải) cho thấy các chấm ít phân tán hơn xung quanh đường y bằng x so với Home Goals (Bên trái). Điều này giải thích cho chỉ số R-squared (R^2) cao hơn đáng kể của mô hình dự đoán bàn thắng đội khách (0.429 so với 0.355).

7.4 Conclusion

7.4.1 Model Summary and Key Finding

Chương này đã thành công trong việc xây dựng và huấn luyện mô hình Hồi quy Poisson (Poisson Regression) để giải quyết bài toán dự đoán dữ liệu đếm (số bàn thắng) trong bóng đá. Dựa trên các chỉ số đánh giá, mô hình này đã đạt được hiệu suất ổn định và tốt nhất cho bài toán này:

- Tính Hiệu quả: Mô hình đạt chỉ số R-squared (R^2) cao nhất là 0.429 cho dự đoán bàn thắng đội khách, chứng minh tính phù hợp thống kê của Poisson Regression với bản chất dữ liệu số bàn thắng thực tế.
- Độ Chính xác: Độ lệch tuyệt đối trung bình (MAE) thấp, chỉ 0.713 bàn, cho thấy dự đoán của mô hình rất gần với kết quả thực tế.
- Hạn chế: Phân tích trực quan cho thấy mô hình Poisson có xu hướng dự đoán quá thấp (underpredict) các tỷ số cao (Actual Goals 5). Điều này chỉ ra rằng mô hình còn mang tính chất bảo thủ, gặp khó khăn trong việc nắm bắt các sự kiện ngoại lai (high-scoring outliers) có xác suất thấp.

CHƯƠNG 8 : ANALYSIS TOOL (CÔNG CỤ PHÂN TÍCH)

8.1 Visualize (Trực quan hoá)

Trong khoa học dữ liệu, trực quan hóa dữ liệu là bước quan trọng giúp hiểu rõ tập dữ liệu, phát hiện xu hướng, và truyền đạt kết quả phân tích. Hai công cụ phổ biến hiện nay là **Tableau** và **Python**.

Mặc dù cùng hướng tới một mục tiêu, nhưng cách tiếp cận, khả năng thao tác dữ liệu và tốc độ thực hiện lại rất khác biệt.

8.2 So sánh tổng quan

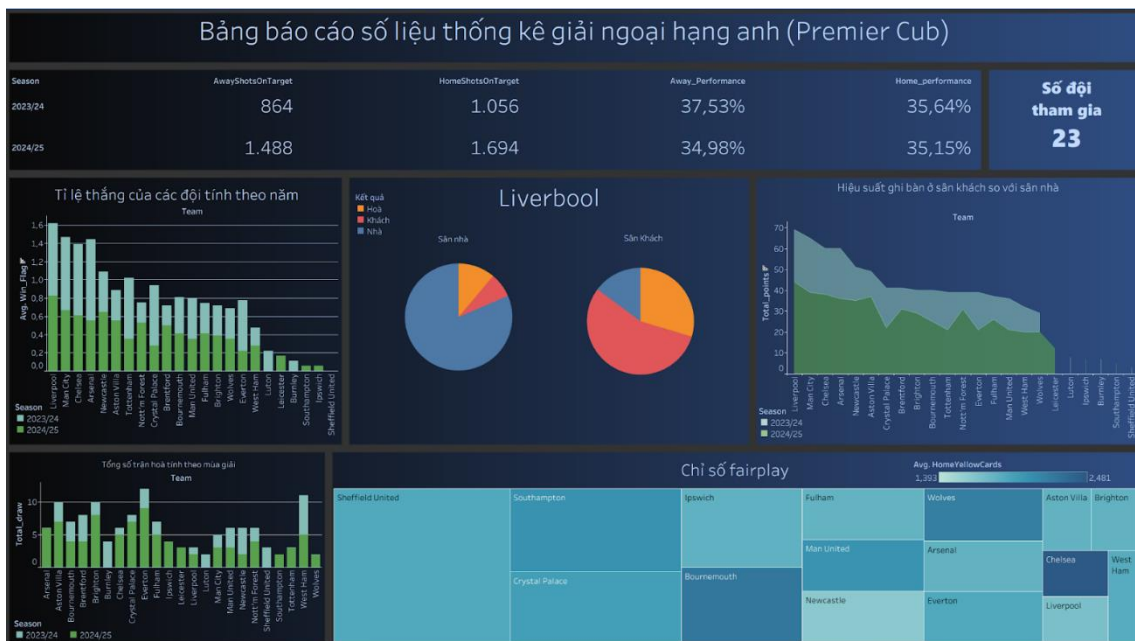


Figure 8.1: Tableau Dashboard for analysis report

8.2.1 Ưu điểm nổi bật của Tableau (so với Seaborn)

8.2.1.1 Không cần lập trình

- Chỉ cần kéo – thả (Drag & Drop) để tạo biểu đồ.
- Phù hợp cho sinh viên, nhà phân tích kinh doanh, người không có kỹ năng code.

8.2.1.2 Tạo biểu đồ rất nhanh

- Từ file Excel/CSV → kéo → có thể tạo Bar Chart, Line Chart, Heatmap trong vài giây.
- Trong khi Seaborn phải viết code `import seaborn as sns, sns.barplot(...)`.

8.2.1.3 Dashboard tương tác & Storytelling

- Kết hợp nhiều biểu đồ → tạo **Dashboard**.
- Cho phép người xem **tương tác, lọc dữ liệu theo thời gian, đội bóng, cầu thủ**.
- Seaborn chỉ tạo biểu đồ tĩnh — không thể bấm lọc trực tiếp.

8.2.1.4 Kết nối trực tiếp với dữ liệu sống (Real-time)

- Kết nối tới file Excel, Google Sheet, SQL Server, MySQL...
- Khi dữ liệu cập nhật → biểu đồ tự động cập nhật.
- Seaborn chỉ đọc file CSV/Python DataFrame tĩnh → phải chạy lại code nếu dữ liệu đổi.

8.2.1.4 Xuất báo cáo đẹp & chuyên nghiệp

- Xuất ra PDF, PowerPoint, file ảnh độ phân giải cao hoặc publish lên web Tableau Public.
- Seaborn xuất ảnh đơn giản, cần chỉnh thủ công nếu muốn trình bày đẹp.

8.2.2.1 Điểm mạnh của Seaborn (để bạn so sánh công bằng)

- Hoàn toàn tùy chỉnh (Điều chỉnh từng chi tiết bằng code (màu sắc, kiểu line, annotation))
- Tự động hóa (Viết script → chạy với dữ liệu mới không cần thao tác thủ công)
- Phù hợp phân tích chuyên sâu (Kết hợp machine learning, regression, statistical test)
- Có thể tích hợp với Pandas, NumPy, Scikit-learn để thực hiện phân tích thống kê và mô hình hóa.
- Tái lập kết quả (Chỉ cần chia sẻ code là tái sử dụng được trên máy khác)

8.3 Khi nào nên dùng Tableau — Khi nào dùng Seaborn?

8.3.1 Tableau

- Làm báo cáo nhanh, trình bày với giảng viên/doanh nghiệp
- Tạo Dashboard tương tác
- Muốn trực quan đẹp mà không cần code

8.3.2 Python (Seaborn)

- Phân tích máy học, viết code quản lý dữ liệu
- Cần script tự động xử lý nhiều file

8.4 Kết luận

Tableau và Seaborn đều là công cụ trực quan hóa dữ liệu hiệu quả, nhưng phục vụ các đối tượng và mục tiêu khác nhau.

- Tableau vượt trội khi yêu cầu về tốc độ, tính trực quan, dashboard tương tác và trình bày kết quả chuyên nghiệp.
- Seaborn phù hợp hơn trong phân tích chuyên sâu, kết hợp mô hình hóa, và khi người dùng cần kiểm soát hoàn toàn bằng code.

Trong bối cảnh làm báo cáo, trực quan hóa dữ liệu bóng đá hoặc trình bày dữ liệu cho doanh nghiệp, Tableau là lựa chọn tối ưu.